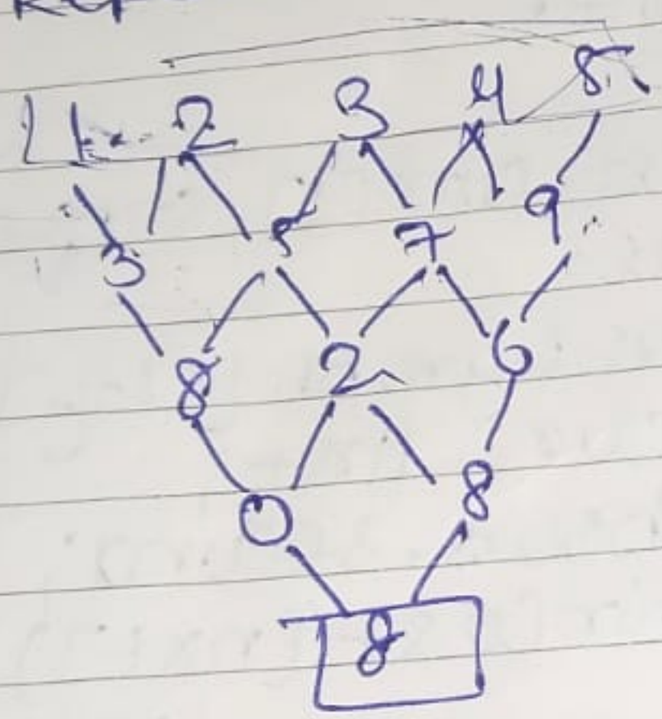
Code :- int[] Printe (int[]

Date
Page

2. nums - 0- indexed
$0 \le nums[i] \le 9$
Triangular sum :- (integer value)
1) n==1 is end of process
Otherwise create a new 0-indexed integer array newNums of length n-1.
2) For each index i, $0 \le i < n-1$
assign the value of newNums[i] = (nums[i] + nums[i+1]) % 10
3) Replace nums into newNums
4) Repeat the process.



Algo :-
1) Create a new Array of size (nums. size - 1)
2) Do Step 1 while the size of nums != 1
3) The new index element is equal to (nums[i-1][j] + nums[i-1][j]) % 10
4) copy the newNums to original nums
5) Return nums[0];

Code: int minPoints (vector<vector<int>>& pts)

Date
Page

TC: $O(N \cdot \frac{(n-1)}{2})$

SC: $O(N) + O(N)$

```
Code: while (nums.size() != 1)
        vector <int> newNums;
        for (int i=0; i<nums.size()-1; i++)
          {
            newNums[i] = (nums[i-1][j-0]
                +nums[i-0][j+1])%10;
          }
        nums = newNums;
      }
      return nums[0];
```

(3) Given an array of points where points[i] = {xi, yi} represents a point on the x-y plane. Return the minimum number of points that lie on same line.

Code:-

```
int maxPoints (vector<vector<int>>& pt)
{
  int ans=1;
  int n=pt.size();
for(int i=0; i<n-1; i++){
  map<double,int> mp;
   for (int j=i+1; j<n; j++) {
     double x = (double)(pt[j][1]-pt[i][1])/
                (double)(pt[j][0]-pt[i][0]);
     if(pt[j][1]-pt[i][1]<0 && (pt[j][0]-pt[i][0])==0)
        mp[abs(x)]++;
        else
           mp[x]++;
     }
  int temp=0;
  for (auto it: mp)
      temp= max(temp,it.second+1);
  ans=max(temp,ans);
   return ans;
```

Algorithm:-

1) Take each point as starting point
2) For that point calculate slope of other point.
3) Store how many time each slope occurs using map.
4) If multiple points have same slope, they are on same straight line
5) Find the max. count of any slope for that point
6) Add 1 to include point itself.
7) Do this for all points
8) Return final maximum

Name= Abhi Kumar
UID= 23BCS12907
section=KRG-1B

Date
Page

1 int findKcheapest (int n, vector<vector<int>& flight, int src, int dest, int k) {
vector<vector<pair<int,int>> graph(n);
for (auto & flight : flights) {
graph [flight[0]].push_back({flight[1], flight[2]});
queue<vector<int>> q;
q.push ({src, 0, 0});
vector<int> minCost (n, INT_MAX);
minCost[src] = 0;
while (!q.empty()) {
auto curr = q.front;
q.pop();
int city= curr[0], cost = curr[1], stop = curr[2]
if (stops > K) continue;
for (auto& neighbours : graph [city]) {
int nxt = neighbours.first;
int price = neighbours.second;
if (cost+price< minCost [nxt])
{
minCost[nxt]= cost +price;
q.push({nxt, cost+price, stops+1});
}
}
}
return minCost[dst] == INT_MAX?
-1 : minCost[dst];
}

TC- ~~O(V+E)~~ O(EK)
SC- O(V)

Date |
Page |

Que Cheapest distance from source to destination with atmost K stops

Algorithm

1. We use BFS in finding the minimum path as BFS always guarantees the minimum possible path.
2. Create the graph using <vector<pair<int, int>>>
3. Create a queue and push the source with 0 weight used 0 cost to itself.
4. Keep a track of mincost
5. While (!q.empty()) {
   We check every neighbour of curr node in q
   if (cost + price < minCost[nxt]
   {
   minCost[nxt] = cost + price
   }
   }
6. Return mincost[dest]

KCH

Algorithm